MIT 6.390 Introduction to Machine Learning

Lab 8: Representation Learning & Latent Space Evolution

A Deep Dive into Neural Concept Formation

[TEACHING ASSISTANT ANSWER KEY]

Aditya Sengupta, Professor Manolis Kellis, & Professor Shen Shen

Spring 2026

1 Introduction: The Representation Problem

To understand modern Deep Learning, we must first understand the "Representation Problem." In early machine learning paradigms (which we explored in the first half of this course), we explicitly designed hand-crafted features. For example, to classify an image of a dog, a human engineer might write code to detect edges, compute color histograms, and identify shapes like "ears" or "snouts." For text classification, we traditionally rely on **Bag of Words** or **TF-IDF** (Term Frequency-Inverse Document Frequency) vectors, which count the occurrences of specific words in a vocabulary of size V.

While mathematically sound, these raw representations are highly problematic:

1. **High Dimensionality:** A vocabulary might contain 100,000 words. A single document is an incredibly sparse 100,000-dimensional vector.
2. **Lack of Semantic Structure:** In a raw TF-IDF space, the word *"dog"* and the word *"puppy"* correspond to entirely different, orthogonal mathematical axes. The raw data structure has absolutely no concept of synonyms, context, or semantic similarity.
3. **Non-Linear Separability:** Real-world data distributions are heavily entangled. A simple linear classifier (like Logistic Regression) cannot easily draw a hyperplane separating "Science" articles from "Art" articles if their vocabulary usages overlap significantly.

2 Deep Learning as Representation Learning

We often treat neural networks purely as complex **function approximators** $f_\theta : x \to y$ that map inputs to output probabilities. However, the true superpower of deep learning is **Representation Learning**.

Instead of forcing a linear classifier to solve an impossible puzzle in the raw, messy input space, a neural network uses successive layers of non-linear transformations (such as ReLU activations and weight matrices) to geometrically **warp, fold, and squash the data space**.

By the time the data reaches the final hidden layer (the "penultimate" layer) of the network, the chaotic raw input has been mapped into a dense, lower-dimensional continuous vector space. This learned mathematical space is called the **Latent Space** or **Embedding Space**.

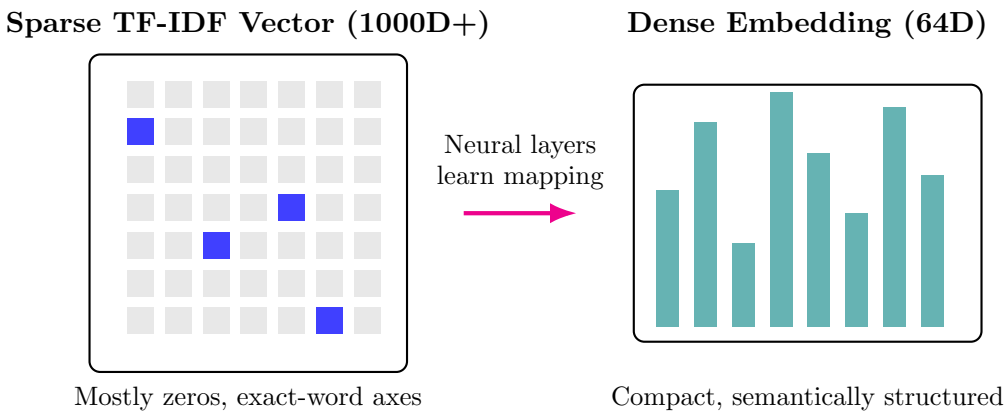

Figure 1: Conceptual transition from sparse TF-IDF features to dense learned embeddings.

In this Latent Space:

- **Proximity represents similarity:** Concepts that are semantically identical (e.g., an article on "Machine Learning" and an article on "Deep Learning") are physically pulled together into tight clusters by the gradient descent process, even if they don't share exact keywords.
- **Linear properties emerge:** The network morphs the dataset to the point where the different categories become perfectly linearly separable. The final *Softmax* classification layer is ultimately just drawing a simple, flat hyperplane boundary through this newly-organized latent geometry.

3 The Dynamics of Learning

Neural networks do not magically know how to construct these beautiful geometric manifolds immediately. In this lab, you will not just look at the final model; you will perform an algorithmic "autopsy" across time.

3.1 Epoch 0: The Chaotic Initialization

Before training begins, PyTorch initializes the weight matrices of your network randomly. If you pass an article through the untrained network, the resulting vector in the latent space is essentially random noise limit. If we plot these untrained embeddings, we will see an interspersed, structureless cloud. The network is completely blind to the underlying concepts.

3.2 Epochs 1 through 10: Gradient Sculpting

As we execute Stochastic Gradient Descent (SGD) and compute the Cross-Entropy Loss, backpropagation begins updating the weights. Mathematically, the gradient acts as an attractive/repulsive physical force in the high-dimensional space:

- **Attractive Forces:** Inputs with the same target label are heavily penalized for being far apart. The network updates its weights to map them closer together.

- **Repulsive Forces:** Inputs with different target labels are penalized for crossing decision boundaries, causing the network geometry to stretch and push contradictory classes away from each other.

As epochs progress, you will physically watch distinct "islands" of knowledge break away from the chaotic center and solidify.

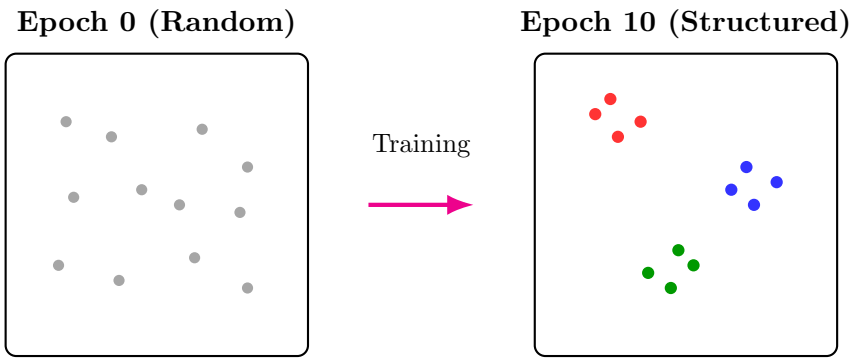

Figure 2: Learning dynamics in latent space: unstructured initialization to clustered concept organization.

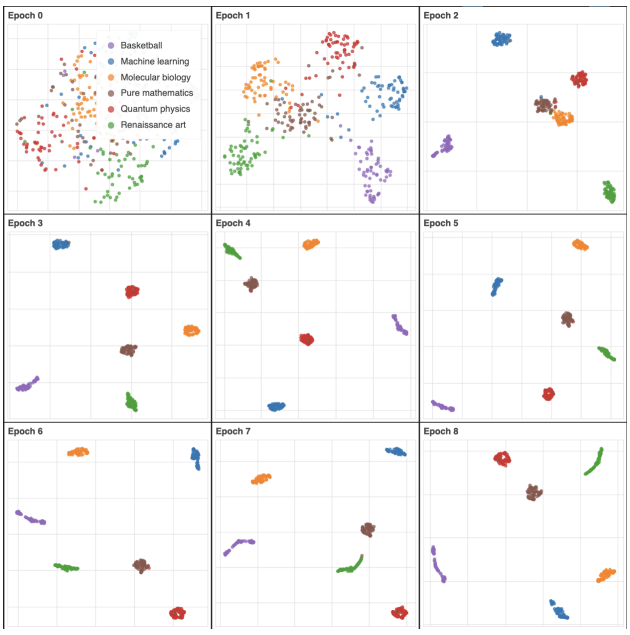

Figure 3: UMAP trajectories across epochs, showing latent-space organization from initialization to trained structure.

4 Visualizing High Dimensions: UMAP & Mantis

A core problem in representation learning is that our latent spaces are typically 64, 128, or 1024 dimensions. Human brains cannot comprehend 128-dimensional geometry.

To solve this, we rely on dimensionality reduction algorithms like **UMAP (Uniform Manifold Approximation and Projection)**. UMAP mathematically preserves the local neighborhood distances of the 128D space while flattening it down to 2D or 3D for us to visualize on a screen.

4.1 The Mantis Platform

To rigorously map the learning trajectory of your model in 3D, you will leverage Mantis, a high-dimensional data visualization and exploration suite extensively used in computational biology and latent mapping at MIT.

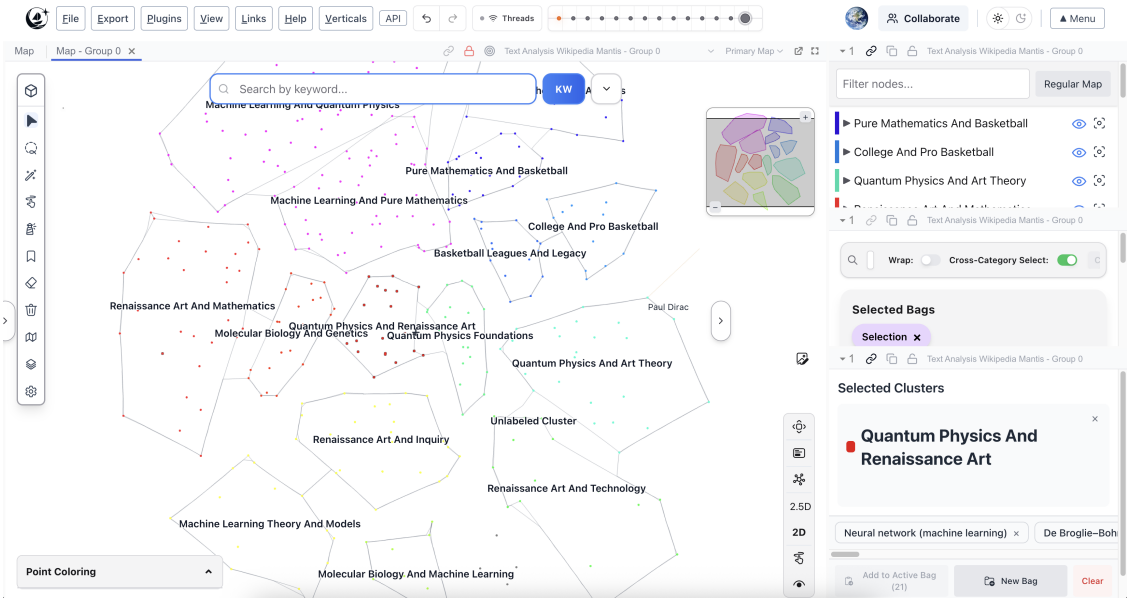

Figure 4: Mantis interface for exploring high-dimensional embeddings projected with UMAP.

5 Lab Activity Walkthrough

5.1 Part 1: Training the Feature Extractor

Your primary objective is to train a multi-layer neural network on a text classification task and freeze snapshots of its latent space at different moments in time.

1. Open the primary lab notebook: `notebooks/part1_text_classification.ipynb`.
2. **The Dataset:** This notebook dynamically queries the real, live Wikipedia API. It searches for highly complex topics (e.g., "Quantum Physics", "Renaissance Art", "Machine Learning") and downloads the first 150 article abstracts for each. *(You are highly encouraged to modify the `topics` list in the Python cell to scrape entirely different subjects that you find interesting!)*
3. **The Baseline Features:** The text is passed through a `TfidfVectorizer` to convert the raw strings into our massive, messy, 1000-dimensional baseline vectors.
4. **The Training Loop:** Execute the training loop. Pay close attention to the fact that we are running the model for 10 *Epochs* (full passes over the dataset).

5.2 Part 2: The Neural Network "Hook"

Study the PyTorch architecture defined in the notebook:

```python
class TextClassifier(nn.Module):
    def __init__(self, vocab_size, hidden_dim, num_classes):
        super(TextClassifier, self).__init__()
        self.fc1 = nn.Linear(vocab_size, 256)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(256, hidden_dim) # Bottleneck representation
        self.classifier = nn.Linear(hidden_dim, num_classes)
        self.embeddings = None # Hook initialization

    def forward(self, x):
        out = self.relu(self.fc1(x))
        out = self.fc2(out) # Shape: [Batch, hidden_dim]
        # ---> THE EXTRACTION HOOK <---
        self.embeddings = out.detach().cpu().numpy()
        out = self.relu(out)
        out = self.classifier(out)
        return out
```

Notice what is happening here! During the `forward` pass, right before the data hits the final `self.classifier` layer, we intercept the `out` tensor. We copy its exact numerical state and save it into `self.embeddings`.

Every single epoch, our loop invokes model evaluations and appends these captured 64-dimensional arrays into a massive list.

5.3 Part 3: CSV Generation and Mantis Upload

Once the model finishes training, the notebook runs a cell to export a widely structured DataFrame containing your raw text, the article titles, and 11 unique columns: `embedding_epoch_0`, `embedding_epoch_1`, ... `embedding_epoch_10`.

1. Download the resulting `text_analysis_wikipedia_mantis.csv` file to your computer.

2. Navigate your web browser to Mantis.
3. Upload your `.csv` file into the platform.
4. In Mantis, select `embedding_epoch_0` as the primary coordinate vector space to render via UMAP. Color the points using the `categoric` metric.
5. Observe the lack of structure.
6. Now, switch the vector space to `embedding_epoch_10`. Watch the points dynamically animate and fly to their newly learned, deeply structured coordinates.

6 Assignment Questions & Solution Key

6.1 Conceptual Representation Questions

Q1 (2 points). Explain in your own words why raw TF-IDF vectors are computationally cumbersome and semantically limited compared to dense neural embeddings.

Solution: TF-IDF vectors strictly score exact word matches. Therefore, a massive 10,000+ length dimension is required just to represent simple vocabularies. Furthermore, because TF-IDF uses orthogonal axes for every single word, it completely fails to capture semantic meaning or context—words like "car" and "automobile" are entirely unlinked in TF-IDF geometry, whereas a neural embedding learns to squash and map both concepts to incredibly close geometric coordinates in dense space.

Q2 (2 points). Why must we explicitly extract and save the latent representations *before* the final softmax or linear classifier layer rather than just looking at the final model prediction probabilities?

Solution: The final softmax or linear classifier layer simply outputs independent class probabilities (e.g. 90% Art, 10% Physics) which tells us little to nothing about the underlying conceptual relationship between those classes inside the model's structural geometry. By extracting the penultimate linear transformations, we capture the high-dimensional representation vector representing the raw semantic features that the neural network "sees". This lets us visualize overlapping semantic meaning rather than mutually exclusive rigid prediction classes.

Q3 (3 points). As training progresses from Epoch 0 to Epoch 10, explain theoretically what transformations you expect to observe regarding the Euclidean distance between a Wikipedia article on "Quantum Physics" and one on "Renaissance Art" versus "Machine Learning" and "Pure Mathematics" within the embedding space?

Solution: As training minimizes the cross-entropy loss, the gradient mathematically pushes data samples belonging to different target categories apart while pulling samples from the same category into tight overlapping centroids. Therefore, the Euclidean gap between "Quantum Physics" and the completely unrelated "Renaissance Art" points will grow massively. Conversely, vectors for "Machine Learning" and "Pure Mathematics" may actually end up physically close to each other or forming bordering clusters rather than totally repelling, because their underlying language structures, equations, and vocabularies are heavily correlated mathematically.

Q4 (2 points). In Epoch 0, the model weights are instantiated randomly. What does the distribution of points look like on the UMAP reduction grid, and conceptually, are there any inherent structures visible before backpropagation?

Solution: The UMAP graph typically looks like a chaotic, overlapping, undifferentiated cloud of noise because the initial random PyTorch weights matrix projects the TF-IDF feature space randomly across multiple dimensions. There is generally zero inherent semantic structural clustering, meaning points for completely different categories are uniformly distributed and mixed up throughout the initial vector space.

6.2 Coding and Architecture Questions

Q5 (3 points). We use a custom PyTorch model class to extract and save the embeddings inline via `self.embeddings = out.detach().cpu().numpy()`.
Why is the `.detach()` method absolutely critical in this assignment? What would physically happen to memory and the PyTorch `autograd` (automatic differentiation) graph if this method was omitted inside our multi-epoch loop?

Solution: `.detach()` separates the tensor cleanly from PyTorch's intensive computational and gradient tracking graph. Without it, the `embeddings_per_epoch` array would unknowingly maintain references to the entire backward-propagation history tree generated by PyTorch in every single training loop pass! Over 11 epochs, storing 300+ full computational graphs and parameter references into RAM would rapidly trigger a catastrophic Out-Of-Memory (OOM) fatal error. You must strip the historical autograd memory to save just the plain numpy matrices.

Q6 (2 points). If a student sets an aggressively high learning rate (e.g., $lr = 0.5$ via the Adam Optimizer) in the Colab notebook, how might the plotted UMAP space behave across Epoch 1 through Epoch 5 in contrast to a well-behaved network using the default $lr = 0.001$?

Solution: When using an excessively huge learning rate, the network's gradient updates "overshoot" the minimum loss landscape violently instead of steadily descending it. The student would see the UMAP clusters thrashing or teleporting wildly across the visualizer grid at random. Instead of cleanly forming tight conceptual islands that smoothly morph from chaos into steady structure, the geometry would oscillate and fail to capture any semantic distinctions cleanly.

Q7 (2 points). When fetching the Wikipedia API dynamically, you can easily change the subjects. Can you think of two Wikipedia topics that might "fool" the neural network into creating overlapping latent clusters at Epoch 10, instead of cleanly separated islands? Briefly justify your answer.

Solution: [Answers will vary but should describe heavily overlapping lexicons]. For example: *"Astronomy"* and *"Astrophysics"*. Because the actual words used in both domains (stars, galaxy, gravity, telescope, observation) are extremely similar, the TF-IDF inputs are nearly identical. The neural network will struggle to separate these two classes significantly without a much deeper architecture, leading to messy, blurred boundary manifolds in the UMAP plot where points merge.

End of Answer Key.